

Bachelor Mathematics  
Track: Computer-Assisted Proofs

---

# Four Colour Theorem

---

by  
Andrea, Abel, Tomas, Lex, Jordy  
February 3, 2023



Department of Mathematics  
Faculty of Sciences



# 1 Introduction

In 1852 Francis Guthrie was coloring in a map of the United Kingdom, when he noticed that to color in any map, he could use as little as four colors. This observation reached mathematician Augustus De Morgan, who in turn shared it with the mathematical world. The Four-Color Theorem emerged: a conjecture stating that to color in any figure with regions, where regions sharing a common boundary are colored differently, four colors would suffice.

Over the next couple of years interest in the conjecture died down, until Alfred Kempe found an article inquiring if the problem had been proven. Only a year later, in 1879, the conjecture was considered to have been solved. Ten years later an error was discovered, the proof holding only for the case of five colors. Nevertheless progress had been made and the techniques developed by Kempe would still be used in later proofs.

In 1913, George Birkhoff published a paper which was on the reducibility of maps in which he improved ways of determining if a *configuration* can appear in a smallest counter example for the theorem. The progress stagnated however as Wernicke (1904), Franklin (1922), and Lebesgue (1940) were the only ones that released a paper on this subject for a while before the idea came to use a computer to do calculations that would take too long manually. Dürre was the one that, under the instructions from Heesch, used a computer program to test if something was recolorable and as such, multiple groups would end up independently trying to prove the four-color theorem through reducibility. It took over a hundred years, but a solution was found in 1976 by Kenneth Appel and Wolfgang Haken, who used their knowledge of programming to efficiently search for reducible configurations, and ended up with an *unavoidable set* of 1482 configurations that are reducible.

The proof was not initially believed by everyone; the program needed to be proven to work. There were many attempts at disproving the theorem, but these were all found to be incorrect. Another point of criticism was that the proof method was disproving all the possible counterexamples that could be made, which couldn't be done by hand (thus one needed to trust the computer); moreover, the non-computer assisted part of the proof was also called into question as it was particularly lengthy, and thus could contain human errors.

## 2 Theory

Before we begin with the Kempe chain games, we must first introduce the required knowledge, as our definition of *D-reducibility* is quite convoluted. The essence of the proof is that if we assume the existence of a counterexample, then it must contain at least one of 2822 specific configurations, as shown by [3]. Furthermore, it has also been proven by an infinite descent argument that a smallest counterexample cannot contain any *D-reducible* configurations. Thus, if we show that all of the above mentioned configurations are D-reducible, then the four color theorem holds.

### 2.1 Required Information

First we begin by noting that any graph used in this report is assumed to be planar; this just means that there is a way to draw the graph such that no two lines intersect. Thus we begin by the most essential definition; that of a *configuration*. A *configuration*  $\mathcal{C}$  is simply a graph that is regular, in the sense that it is nonempty, connected, contains no bridges and no vertex of degree 1 and any two distinct regions have at most one common borderline. Moreover, the outer vertices of a configuration form a ring  $\mathcal{R}$  that is of size greater than 5; and lastly bounded faces have triangular borders, i.e. every triangle is a border of a face. The simplest example is in Figure 2.1.

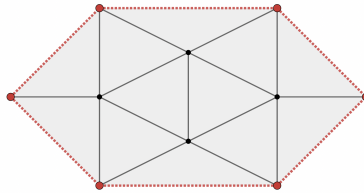


Figure 2.1: The Birkhoff diamond was the first *reducible* configuration to be identified and is the smallest of its kind.

The concept of a *ring* is particularly important; in Figure 2.1 we indicate it as the subgraph colored in red. To get a more in depth view about the theorem, some explanations are needed; like that of the *Kempe chain*, which is a two-colored chain of vertices, or the *Kempe interchange* where each color of a vertex of a Kempe chain is swapped with that of the respective other color. In essence, a Kempe interchange yields a 4-coloring that is itself admissible. For a valid boundary coloring, say that no two adjacent vertices contain the same color, when two boundary colorings have a bijection such that through a swapping of the numbers the coloring becomes the same, then both colorings can be considered equivalent. Since these equivalent colorings give the same result, only essential colorings are used. An essential boundary coloring is the coloring whose numbering compared to that of other equivalent colorings has the lowest numbers in the front. Use the same method to order all essential boundary colorings of the ring in the lexicographic order.

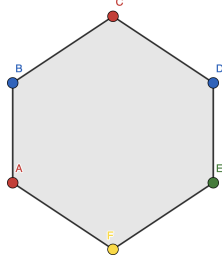


Figure 2.2: Example of an essential boundary coloring  $c$  of a ring of size 6. Notice the Kempe (red,blue)-chain.

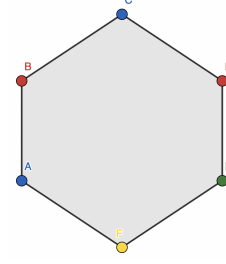


Figure 2.3: The admissible 4-coloring  $c'$  produced by a Kempe interchange on  $c$ .

To talk about the method of finding the reducibility, it is needed to explain about the two sets of coloring used. The set of essential colorings of the ring  $\mathcal{R}$  of size  $r$  is denoted by  $\Phi(r)$ , without taking into consideration the inner part of configuration,  $\mathcal{C}$ ; while  $\Phi_0(\mathcal{C})$  is the set of essential colorings that are called *directly color-extendible*, or *good from the outset*, of a configuration  $\mathcal{C}$ ; i.e. these colorings do not require any Kempe interchanges in order to extend to a valid coloring of the configuration.

The amount of essential colorings of any ring  $r$  can be calculated using the following;

$$f(r) = \begin{cases} \frac{3^{r-1}-1}{8} & \text{for } r \text{ odd} \\ \frac{3^{r-1}+5}{8} & \text{for } r \text{ even} \end{cases} \quad (2.1)$$

shown in page 188-189 of [1].

## 2.2 Blocks, and block decompositions

We want to create a way to properly recolor any boundary coloring. To do this we need to work up to what is called a *block decomposition*.

**Definition 2.1.** Consider a partition of four colors into two pairs. If we take a color-pair from the partition, for instance  $\{0, \omega\}$ , we can find Kempe chains containing only vertices of the colors 0 and  $\omega$ . If such a chain has maximal length, meaning it cannot be extended within the ring, we call it a *Kempe sector*.

**Definition 2.2.** A partition of the four colors allows us to partition the vertices of our ring into Kempe sectors. When we group together some sectors of the same type, meaning the sectors can only contain vertices of the same color partition, we obtain a *block*. The rules of constructing these blocks are specified in theorem .

**Definition 2.3.** By taking a walk on the whole ring and number the sectors in order of appearance, we can construct the *index set*  $S = \{1, 2, \dots, m\}$ , for a boundary coloring with  $m$  Kempe sectors. Thus every sector is represented by a natural number.

**Definition 2.4.** Given a partition of the index set  $S = \{1, \dots, r\}$  into blocks  $B_1, \dots, B_s$ , we say that the blocks  $B_x$  and  $B_y$  *abut* one another if there exists an edge that connects the two blocks together. In other words two blocks are abut if there is a sector  $p \in B_x$  that lies directly next to a sector  $q \in B_y$ . In the case where  $p < q \in S$  then we say that  $B_x$  *directionally abuts*  $B_y$ . Note that  $p + 1 = q$ .

Consider the following theorem from [1]:

**Theorem 6.6.4.** *Let  $G'$  be a colored connected graph without bridges or final edges all of whose faces, other than its unique exceptional face, are bordered by triangles. Suppose a boundary coloring  $a = (a_1, \dots, a_r)$  of size  $r$  and a color partition  $\omega \in \{1, 2, 3\}$  are given. Then a partition of the index set  $S = \{1, \dots, r\}$  into blocks  $B_1, \dots, B_s$  is a block decomposition (with respect to  $a$  and  $\omega$ ) if and only if:*

1. *Each block is a union of Kempe sectors of the same type. Notation:  $\omega$ -block, respectively  $\bar{\omega}$ -block.*
2. *Blocks cannot mutually overlap. In other words, for  $k_1, k_2 \in B_k$  and  $l_1, l_2 \in B_l$ ,  $k \neq l$ , the order*

$$k_1 < l_1 < k_2 < l_2$$

*is not possible.*

3. *If two blocks abut at one point, then they abut at exactly one more point.*

**Theorem 2.5.** *Theorem 6.6.4 from [1] is equivalent to the following statements:*

- a. *Each block is a union of Kempe sectors of the same type.*
- b. *If two blocks abut at one point, then they must abut in exactly one more point in the opposite direction. In other words, if two blocks directionally abut by  $p \in B_x$  and  $q \in B_y$ , meaning  $p + 1 = q \in S$ , there exist unique  $p' \in B_x$  and  $q' \in B_y$  such that  $q' + 1 = p'$ .*
- c. *Create  $S' = \{p, q, \dots, n\}$  by shifting  $S = \{1, 2, \dots, r\}$  such that the first entry of  $S'$  is  $p$ . Then for all  $i \in \{q, \dots, q'\} \subset S'$  we have  $i \notin B_x$ . Likewise, for all  $j \in \{p', \dots, n\} \subset S'$  we have  $j \notin B_y$ .*

*Proof.* For length's sake, we will only give a broad outline of the proof of this theorem. The first property is the same for both sets of properties. The equivalence of “3” and “b” holds as otherwise if we have two blocks abutting twice but not following our directional property, then that would be contradicting rule “2” and rule “c” respectively. The remaining properties are more or less trivial, and are not difficult to prove.  $\square$

**Definition 2.6.** Let  $S$  be the set of Kempe sectors of a boundary coloring and let  $B_1, \dots, B_n$  be a given (unordered) block decomposition. Let  $f$  be a function that sends

each sector to its corresponding block. We define the *lexicographic block decomposition function*  $f^*(x) : S \rightarrow \{1, \dots, n\}$  recursively:

$$f^*(x) = \begin{cases} 1 & \text{if } x = 1, \\ 2 & \text{if } x = 2, \\ f^*(y) & \text{if there exists a } y < x \in S \text{ such that } f(y) = f(x), \\ \max(f^*(y) \mid y < x) + 1 & \text{if for all } y < x \text{ we have } f(y) \neq f(x). \end{cases}$$

**Definition 2.7.** Let  $S = \{1, \dots, m\}$  be an index set of Kempe sectors and let  $L = (f^*(S)) = (f^*(1), f^*(2), \dots, f^*(m))$  be a list of block indices corresponding with a block decomposition. Let  $i \in S$ . We define the *arrow function*  $g : L \rightarrow \{\uparrow, \downarrow\}$  as follows:

$$g(f^*(i)) = \begin{cases} \uparrow & \text{if } f^*(i) < f^*(i+1), \\ \downarrow & \text{if } f^*(i) > f^*(i+1) \text{ or } i = m. \end{cases}$$

**Theorem 2.8.** Let  $S, L$  and  $g$  be defined the same as above.

1. The list  $L = (f^*(1), f^*(2), \dots, f^*(m))$  corresponding with a block decomposition, has a unique representation of arrows  $\mathcal{A} = (g(f^*(1)), g(f^*(2)), \dots, g(f^*(m)))$ .
2. A list  $\mathcal{A}$  of  $\uparrow$ 's and  $\downarrow$ 's corresponds to a unique list  $L = (f^*(1), f^*(2), \dots, f^*(m))$  if and only if  $\mathcal{A}$  abides by these two rules:
  - i. The amount of  $\uparrow$  and  $\downarrow$  given a correct set of sectors  $S$  will always be equal.
  - ii. The image of any truncation  $\{1, 2, \dots, J\} \subseteq S$  will have more or equal amount of  $\uparrow$  than  $\downarrow$ .
3. As a corollary, we can define our aforementioned correspondence to be the inverse of  $g$ , this implies that our function  $g$  is a bijection.

*Proof.* Once more for the sake of length, we will give a short summary of the proofs rather than the full proofs. It is easy to note that since  $g$  has an obvious inverse, property 3 holds. Next,

1. There can only be an even amount of Kempe sectors, now using induction on the amount of Kempe sectors (with step-width 2) we have been able to prove that we will indeed always get an unique representation of  $\uparrow$  and  $\downarrow$ 's.
2. This proof is pretty long and tedious, the idea comes down to the following: In order for  $\mathcal{A}$  abide by the rules of a block decomposition in Theorem 2.5, it can be shown that every  $\uparrow$  creates a new type of block type and every  $\downarrow$  implies that the next sector needs to be in the same block as the block in front of the first instance of a sector that is in the same block as the sector before the  $\downarrow$ . For clarification let us look at the following example. Let  $\mathcal{A}$  be:  $(\uparrow, \uparrow, \downarrow, \uparrow, \downarrow, \downarrow)$ .  $\mathcal{A}$  follows both “a” and “b”, thus there should be a corresponding correct block decomposition. Abiding by the aforementioned rules for  $\uparrow$  and  $\downarrow$  we will get the following list  $L = (1, 2, 3, 2, 4, 2)$  This list is not only the unique possibility, but also follows the properties of Theorem 2.5; thus it corresponds uniquely to a block decomposition.

□

## 2.3 Kempe Chain Games

Essentially, D-reducibility is achieved when for all block decompositions for all of the boundary colorings we can find a series of Kempe interchanges that turn the initial coloring into a coloring which is *good from the outset*. This is what the following components of the Dürre-Heesch algorithm look to confirm. We will use the Birkhoff diamond as an example to demonstrate the next steps of the algorithm. The calculations up to now have determined that boundary coloring number 1,  $(0,1,0,1,0,1)$ , cannot be directly extended; thus we will now determine which Kempe interchanges are available. Notice that the selection of the color-pair  $\omega = 1$  does not yield any results, as we are only able to obtain one Kempe sector rendering the only possible Kempe interchange to yield the nonessential boundary coloring  $(1, 0, 1, 0, 1, 0)$ .

Figure 2.4: Block decompositions arising when taking the coloring  $(0,1,0,1,0,1)$  and any partition other than  $\omega = 1$ .

| No. | $B_1$   | $B_2$   | $B_3$     | $B_4$       |
|-----|---------|---------|-----------|-------------|
| 1.  | $\{1\}$ | $\{3\}$ | $\{5\}$   | $\{2,4,6\}$ |
| 2.  | $\{2\}$ | $\{4\}$ | $\{6\}$   | $\{1,3,5\}$ |
| 3.  | $\{1\}$ | $\{4\}$ | $\{2,6\}$ | $\{3,5\}$   |
| 4.  | $\{2\}$ | $\{5\}$ | $\{1,3\}$ | $\{4,6\}$   |
| 5.  | $\{3\}$ | $\{6\}$ | $\{1,5\}$ | $\{2,4\}$   |

The analysis of boundary coloring 1 reveals that choices of partition  $\omega = 2$  and  $\omega = 3$  must be equal. The computer can then determine the next possibility, with  $\omega = 2$ . This results in six Kempe sectors, each containing a single vertex. There are five different possible block decompositions, all with 4 distinct block types, as shown in the table above. For each of these decompositions, it is possible to apply a Kempe interchange which leads to a boundary coloring that is good from the outset. To test this, a step-by-step approach is taken, starting with one block decomposition and progressing to the next. We begin by showing this for decomposition 1 and 5;

- i. Apply a Kempe interchange on  $B_1$  by replacing color 0 with color 2, then giving the corresponding essential coloring; more precisely, the steps are  $(0,1,0,1,0,1) \rightarrow (2,1,0,1,0,1) \rightarrow (0,1,2,1,2,1)$ ; which is an essential boundary coloring (lexicographically it is the 21st), and is good from the outset by color-extendibility calculations. Hence, we are done with this block decomposition.
- v. In this case, one may verify that we need to make more than one recoloring to complete the game. This game will consist in replacing color 0 with color 2 in  $B_1$ , and simultaneously replacing color 1 with color 3 in  $B_2$ . The game steps are  $(0,1,0,1,0,1) \rightarrow (0,1,2,1,0,3)$ ; which can be shown to be good from the outset.

Once we are done with all possible block decompositions, for this specific essential coloring, i.e.,  $(0,1,0,1,0,1)$ , it follows that this coloring of the Birkhoff diamond is *good of*

*stage 1.*

In other words, an essential coloring of a configuration  $\mathcal{C}$  is *good of stage 1* if, while not good from the outset, the Kempe chain game may be played on any block decomposition in a way that after any single Kempe interchange, we may obtain a coloring that is good from the outset; more precisely, the coloring obtained is an element of  $\Phi_0(\mathcal{C})$ . We denote by  $\Phi_1(\mathcal{C})$ , the collection of colorings of the configuration  $\mathcal{C}$  that are good of stage 1, and those that are good from the outset.

For a given boundary coloring, color-pair choice, and block decomposition, a Kempe interchange or, a move in the Kempe chain game is determined by the representation of the blocks in which the interchange takes place. Consequently, the number of possible moves is finite and is equal to the number of block selections. However, equivalent results can be achieved through different selections; for example, if an interchange is completed within all  $\omega$ -blocks, then the resulting coloring is equivalent to the original. The same effect occurs if the interchange is conducted within all  $\bar{\omega}$ -blocks or if it is done on a global scale, across all blocks. With all the above in mind, we define the concept of  $\Phi$ -goodness.

**Definition 2.9.** Let  $\Phi(r)$  be the set of boundary colorings of size  $r$ . A boundary coloring  $a \in \Phi(r)$  is  $\Phi$ -good, or has *goodness class* equal to 1, if it does not belong to  $\Phi$  itself, but there exists a color-pair choice  $\omega$  such that for all corresponding block decompositions a Kempe interchange exists that turns  $a$  into an element of  $\Phi$ .

Note that since Kempe interchanges do not usually preserve essentiality, we can assume that  $\Phi$ , above, is closed under equivalences. Furthermore, we obtain that the property of  $\Phi$ -goodness is also closed under equivalences. Therefore, for any coloring  $c$  of a ring of size  $r$  that is equivalent to a coloring that is an element of  $\Phi$ , we have that  $c$  is an element of  $\Phi$  itself. It is useful to note, that in our case of the Birkhoff diamond, those colorings that are  $\Phi$ -good, are precisely those that are elements of  $\Phi_1(\mathcal{C})/\Phi_0(\mathcal{C})$ . One further way we will refer to these elements is as *good of stage 1*, where 1 clearly denotes the number of Kempe interchanges. More generally;

**Definition 2.10.** Let  $\mathcal{C}$  be a configuration,  $n$  a natural number, and  $\Phi_n(\mathcal{C})$  the set of already established boundary colorings that are of goodness class less than or equal to  $n$ . A boundary coloring is *good of stage  $n + 1$*  if it is  $\Phi_n(\mathcal{C})$ -good; in other words, with exactly  $n + 1$  Kempe interchanges it can be transformed into a  $\Phi_0(\mathcal{C})$ -good.

Essentially, we are constructing an increasing chain of sets of boundary colorings in  $\Phi_n(\mathcal{C})$ . Since  $\Phi(r)$  is a finite set, there must exist  $\tilde{n} \in \mathbb{N}$ , such that  $\Phi_N(\mathcal{C}) = \Phi_{N+1}(\mathcal{C}) = \dots$ . We define  $\bar{\Phi}(\mathcal{C}) = \Phi_N(\mathcal{C})$ , as the outcome is inherently independent of the actual value of  $N$ . Finally, we come to the final, and most important definition;

**Definition 2.11.** The configuration  $\mathcal{C}$  is said to be *D-reducible* if  $\bar{\Phi}(\mathcal{C}) = \Phi(r)$ ; i.e. all boundary colorings are of some goodness stage. In other words, for all boundary colorings, we can find finite Kempe interchanges that transform each to one that is good from the outset.

Applied to our longstanding example, it is simple to calculate (much faster if done through a computer) that for the Birkhoff diamond,  $\overline{\Phi} = \Phi_5 = \Phi(r)$ . More importantly, we obtain that our Birkhoff diamond is a *D-reducible* configuration.

## 3 Computer-Assisted Proof

In order to describe the challenges that went into the code, it is important to reiterate exactly what we are trying to prove. A configuration is *D-reducible* when every essential coloring of the ring around said configuration is good. Goodness is recursively defined: Every extendable coloring is good, and any other coloring is good when there exists a color-partition (to be referred to as just “partition” from now on) such that for all block decompositions induced by said partition, there exists a Kempe-Interchange that leads to a good coloring. If one can’t prove a coloring to be good based on these two facts, then it’s not good. This definition leads us to an “obvious” algorithm to determine if a configuration is D-reducible.

### 3.1 Naive Approach

The very first algorithm we programmed for this project works like this (it is somewhat simplified to save space):

- Step 0 : Before the algorithm even begins, it computes all essential colorings for all ring-sizes (6 being the minimum and 16 being the maximum).
- Step 1 : It creates a list of good colorings (which starts out empty), goes through the list of all essential colorings, checks if they are extendable, and if so, adds them to the list of good colorings.
- Step 2 : It goes through the list of all colorings again, skipping over ones already declared to be good.
- Step 3 : For all of these colorings, it goes through all 3 possible partitions.
- Step 4 : It generates the block decompositions based on the current coloring and partition, and goes through all of those.
- Step 5 : It generates the list of all colorings you can get to from the current coloring via a Kempe-interchange using the current partition and block decomposition, and goes through those.
- Step 6 : Checks if those colors are in the list of good colorings. By sending information back through these nested loops we can ensure that ...

Step 7 : ... if there exists a partition such that for all block decompositions there exists a Kempe-interchange leading to a good coloring, it adds the current coloring to the list of good colorings.

Step 8 : It repeats steps 2-7 until either the amount of good colorings has not increased (in which case it returns false) or the amount of good colorings equals the total amount of colorings (in which case it returns true)

### **What needs to be faster, and what does not need to be faster:**

The previously described algorithm works, but is very slow. It struggles with even ring-size 11 configurations. In order to improve it we must identify what causes the slowness. First, there are some programming-specific optimizations that, while do help, do not change the algorithm. As an example, it is faster for a program to note down that it should skip over an element of a list in the future, than it is to remove said element from the list permanently. This is because the latter requires the code to reinitialize the list, while the former does not. For the sake of time, other than this brief mention, we will only focus on the more interesting and involved optimization steps.

First, Step 0 we do not need to improve. It takes maybe a minute for it to complete, and it only has to do it once. The same applies to Step 1, it takes 4 and a half minutes for the largest ring-size, and considerably less for all others. Steps 2, 3, 7 and 8 are all essential parts of the algorithm that cannot be changed without fundamentally changing everything else about the algorithm. This, by process of elimination, leaves steps 4-6 as the primary points where improvement is necessary.

### **How we improved things:**

First, we focused in on the single slowest part: Step 6. In our original algorithm, this step amounts to checking if an item is in a list. This generally is very fast, but notice that it is the deepest part of a nested-loop: the algorithm performs step 6 orders of magnitude more times than it performs any other step. To improve step 6, the following fact comes in handy: storing objects in a list is very efficient for the purpose of going through every element of said list in a for-loop. It is however, distinctly inefficient (arguably the least efficient way of storing data) if the goal is purely to locate elements within that list. For that purpose, it is better to use a tree data structure. As we never have to loop through the set of good colorings in our algorithm - only check if a coloring is in it - this is perfect for our purposes. Instead of formally explaining the construction of the tree, it is easier to just use an example: if the coloring  $[0,1,2,1]$  is in the tree, that means the root node (which always has color 0) has a child with color 1, which has a child with color 2 which has a child with color 1. This essentially reduces the amount of time it takes to check if something is in the set of good colorings to “at most as many steps as the size of the ring”.

The second obvious part that can be improved is Step 4. Right now, it generates every block decomposition every time it gets to a coloring and partition. The first thing to note here is that technically one can do all of this up-front. We could, given infinite memory, generate 3 lists of block decompositions for each coloring and each ring-size upfront,

and just read from those lists. It is, however, not possible in practice due to memory-allocation issues. On the other hand, we don't actually need to do this: many colorings (with partitions) produce the same Kempe-sectors, thus the same block decompositions. Checking what the Kempe sectors of a coloring are with respect to a partition amounts to just checking which of the colors of said coloring are in the color-pair containing 0, and which ones are in the other color pair. There are 2 possibilities for each color (except the 1st), so that means the amount of lists of block decompositions we need to generate is only in the thousands, not millions. These lists can also be stored in a tree, making accessing them as fast as possible.

The only part left to improve after that is step 5. Right now, it just generates all colorings you can get to via a Kempe interchange (and then converts it to an essential coloring). Our idea for optimizing this was simple: Merge steps 5 and 6 into one step, and essentially check for the existence of a Kempe interchange without computing every Kempe interchange. To explain this, we will use the analogy of a Sudoku puzzle: When we try to determine if a Sudoku puzzle is solvable, we do not go through the list of all possible combinations of numbers, and check one by one if it solves the puzzle. Instead, we continuously eliminate possibilities using the rules of Sudoku, until we either arrive at a solution, a point where we have to "guess", or at a contradiction. Our new algorithm works similarly: It has a "possibility list", which is a list of 0s, 1s, or 2s. 0 means "don't change this color", 1 means "change this color", and 2 means "we don't know if this color needs to be changed". Using the rules of Kempe-interchanges it keeps altering this possibility list, guessing when it has to, until it either finds a Kempe interchange leading to a good coloring, or concludes that to be impossible.

There is, however, one problem with this: it, by construction, cannot account for the possibility of an essential coloring changing into a non-essential one, which can lead to false negatives. It rarely does so, but it does it enough times to be worth consideration. To counter-act this, we add an extra step to the end of the whole algorithm, where if it were to return false, it "double-checks" by running the previous, 2nd-best version of the algorithm on the remaining colorings.

### Talking about speed:

| Config, Ring size | Alg 1   | Alg 2  | Alg 3  | Alg 4  |
|-------------------|---------|--------|--------|--------|
| 1, $r = 6$        | 0.007s  | 0.005s | 0.004s | 0.007s |
| 2, $r = 7$        | 0.030s  | 0.033s | 0.023s | 0.034s |
| 3, $r = 8$        | 0.156s  | 0.155s | 0.125s | 0.097s |
| 6, $r = 9$        | 1.338s  | 1.215s | 0.817s | 0.597s |
| 11, $r = 10$      | 11.392s | 7.609s | 5.839s | 3.595s |
| 21, $r = 11$      | 116.15s | 49.54s | 35.75s | 15.53s |
| 27, $r = 12$      | 39m 35s | 6m 17s | 4m 58s | 2m 7s  |

As it can be seen, the optimizations we detailed earlier drastically improve performance. For a ring-size 12 example, it brings computing time down from around 40 minutes to just 2. The table is based on running the algorithm on a home computer checking one

configuration at a time. After tweaking the code to allow for multi-processing we ran it on a high end PC and after twelve hours every configuration was checked and confirmed to be  $D$ -reducible.

**Potential future improvements we were not able to implement:**

If one were to be able to get rid of the “false negative” problem in the final optimization, it would definitely increase speed, as it would be able to confirm many good colorings earlier. We did not do this, as while we are sure it is possible, we did not have time to implement it. Second, with more memory, for each coloring and partition, one could be able to skip through block decompositions with which we already know you can get to a good coloring. This would require 3 lists of 0s assigned to each coloring (as there are 3 partitions to choose from), and whenever we determine a block decomposition for a coloring and partition to be “good”, you change the relevant 0 in the relevant list to a 1. This ensures that when the code gets to this same coloring and partition again, it doesn’t go through the pointless trouble of re-confirming something it should already know.

To access the code in detail, visit [2].

## 4 Conclusion

In this report we have thoroughly outlined an algorithm to show that a configuration is  $D$ -reducible; and once we have shown that all 2822 configurations are as such, the four-color theorem follows by “lack of counter-example”. Furthermore, it must be noted that without the assistance of computers, we likely would have never been able to prove the theorem; for example, for any configuration with a corresponding ring that is of size 16, we would need apply Kempe chain games to around 1.7 million different colorings; a task that, at human speed, would take an immense amount of time.

## Bibliography

- [1] R. Fritsch, G. Fritsch, and J. Peschke. *The Four-Color Theorem: History, Topological Foundations, and Idea of Proof*. Springer, 1998. ISBN: 9780387984971. URL: <https://books.google.nl/books?id=oGuXAkR6t1MC>.
- [2] Github. *BonucciAndrea/DurreHeesch: Dürre-Heesch*. 2023. URL: <https://github.com/BonucciAndrea/DurreHeesch>.
- [3] John Steinberger. *An unavoidable set of  $D$ -reducible configurations*. 2009. DOI: 10.48550/ARXIV.0905.0043. URL: <https://arxiv.org/abs/0905.0043>.